House Price Prediction Using Neural Networks

Incorporating Neighbourhood-Based Features via Graph Representation

---

Module: Graphs, Networks and Systems

Course: BSc Data Science

University: University of Leeds

Group Members:

Abraham Sobowale

Lucy Cullender

Komail Alhelli

Shruthi Rajesh Babu

Temitayo Olowolagba

Problem understanding

The primary objective of this project is to create a predictive model for Californian real estate pricing. This was approached by using a supervised learning regression approach, aiming to predict a continuous target variable: the median house value for certain areas. Property prices are never influenced by isolation; they are significantly influenced by a combination of individual structural characteristics and the broader economic factors of their surrounding neighbourhoods. The core task is to model these complex relationships using a standard artificial neural network. Furthermore, the aim is to determine whether encoding spatial proximity into a simple graph structure can extract impactful input features to improve the model's baseline prediction accuracy.

Methodology

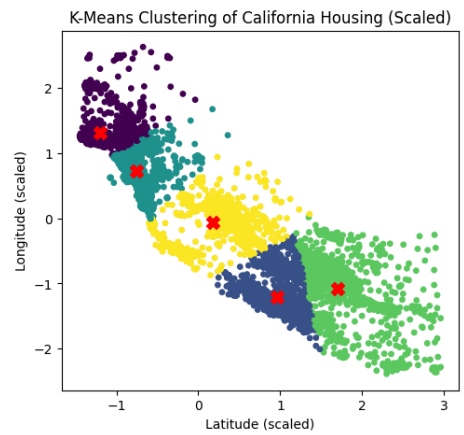
Dataset Description

To train and evaluate the models, California Housing dataset was used **(Pace and Barry, 1997)**, which is publicly accessible with the scikit-learn library. **(Pedregosa et al., 2011)** It is important to note that the data does not represent individual homes. Instead, each data point corresponds to a census block group, acting as a localised housing area in California.

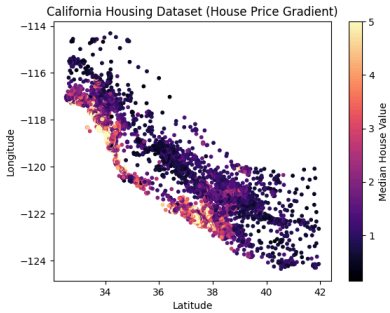
The dataset provides a comprehensive set of input features comprising aggregated housing and population statistics. These numerical metrics include:

- Median income of the residents.
- House Age, average number of rooms and bedrooms per household.
- General population-related statistics for the block group.
- Geographical location (latitude and longitude)

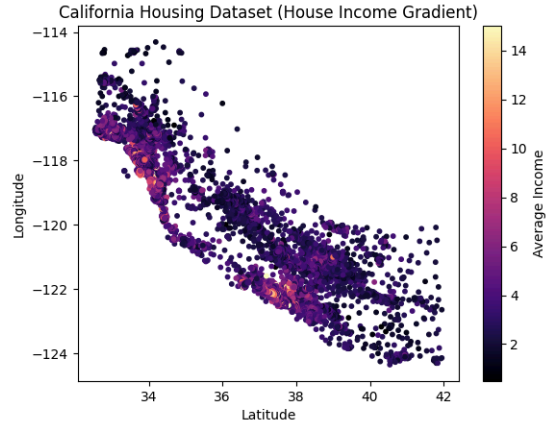
Initial analysis suggests these metrics all carry a significant role in the target house pricing. Using simple visualisation plots alongside clustering and heatmaps, it can better illustrate how impactful these metrics are.



Nearest Neighbours Clusters showing the main 5 housing areas in California (From left to right: Sacramento (rural north area), San Francisco Bay Area (incl. San Jose), Central Valley (Fresno centre), Los Angeles, San Diego (far south area)). Geographically this shows the north is much more rural with the smallest clusters (fewest houses) being in the Sacramento area. In the Centre cluster, the inhabitable strip running through the centre can be seen (which would be the Great Valley, hence the lack of housing in that area). The most popular areas are visualised as the largest, more dense clusters of LA and San Diego regions, where housing is more dense and the populations are larger.



As an initial visualisation of the base-prediction housing prices, it is illustrated that the most expensive properties tend to be closer to the coastline overall, with a larger proportion of the most expensive housing in the Los Angeles area specifically (as expected).



A similar gradient follows for median neighbourhood income as one of the 8 features in this dataset. The Majority of the higher incomes are in Los Angeles again and near the coast in the San Francisco area.

Data Processing

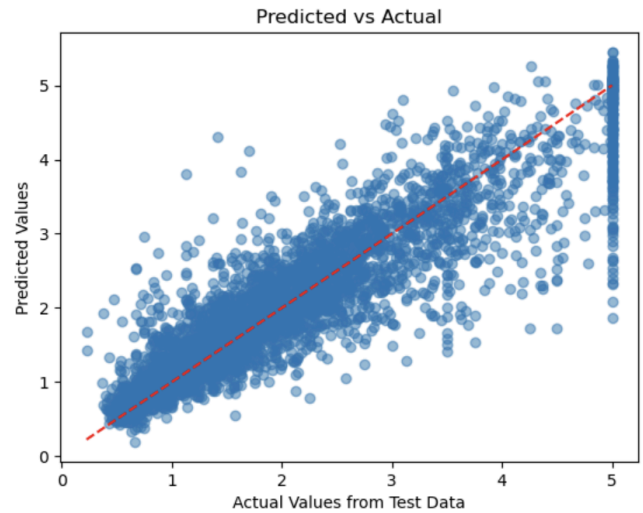
In order to effectively explore the dataset, Pandas was utilised to manipulate the dataset. Sklearn has a prebuilt option to import this directly, allowing for dataset split by the target (median house price) and the input features.

Subsequently, the data was split in order to effectively train the upcoming model. Selecting a low split risks underfitting (the model is too simple to be useful). Conversely, high split risks overfitting where the test size is too small for reliable evaluation. It was decided that a standard 80:20 ***(2024, Buhl)*** random split would be suitable, balancing training with evaluation. This split was randomised using a specific state value to keep results consistent.

Additionally, neural networks are sensitive to the magnitudes of the different metrics used. Using the raw data would cause the algorithm to handle various gradients, hindering convergence. To prevent this, standard normalisation was applied only to the input features. This approach allowed metrics to move on a uniform scale without impacting the train environment.

Baseline Neural Network Model Training

The MLPRegressor performs both forward propagation and back propagation to reduce the error between the predicted and actual outputs. The code uses two hidden layers, 64 and 32. These layers are the core components of the neural networks architecture. Each layer computes $z = WX + b$ where X is each input feature, W is the weight of each feature, and b is the bias. It forms linear combinations of the input variables in the dataset and then models the target as a non-linear activation function which is ReLu in this case as it is good at modelling non-linear relationships and house prices are not linear. The model uses mean squared error (MSE) as the loss function during training, which is standard for regression tasks.



Finally, the metrics computed outside of the model (both R^2 test and mean squared error) compare the predicted output with the actual output to see how well the model performs. The R^2 test shows that the model is neither overfitting nor underfitting, therefore showing that the predicted outputs and actual outputs are similar and well balanced.
The figure below is a plot of the predicted values against the actual values.

The strong positive correlation following the red dashed line confirms what the R^2 test showed, which was that the model is well balanced.
The Mean Squared Error computes the calculation $MSE = \frac{1}{n}\sum(y_{actual} - y_{predicted})^2$ which is a clear comparison of the predicted and actual outputs.

Constructing Neighbourhood-based Features

Feature 1: *Average Neighbour House Price (avg_nb_price)*

A neighbourhood based feature was added to capture spatial relationships between housing areas. This idea came from the assumption that geographically close areas tend to have similar house prices. Using the original latitude and longitude variables, neighbourhood connections were defined through a k-nearest neighbours (k = 10) approach based on Euclidean distance.

To avoid data leakage, the k-Nearest Neighbours model was fitted only on training data coordinates, and the neighbours for both training and test observations were identified exclusively from the training set. Using these neighbour indices, a new feature representing the average house price of neighbouring areas (avg_nb_price) was computed from training target values.

This has resulted in two new variables, one for the train sample and another for the test sample, both of which were derived solely from training price information. Avg_nb_price feature was added to capture local spatial price patterns into the model. This allows the enhanced neural network to better learn geographical dependencies and improve prediction performance compared with the baseline model which only used original features.

Feature 2: *Average Neighbour Median Income (avg_nb_income)*

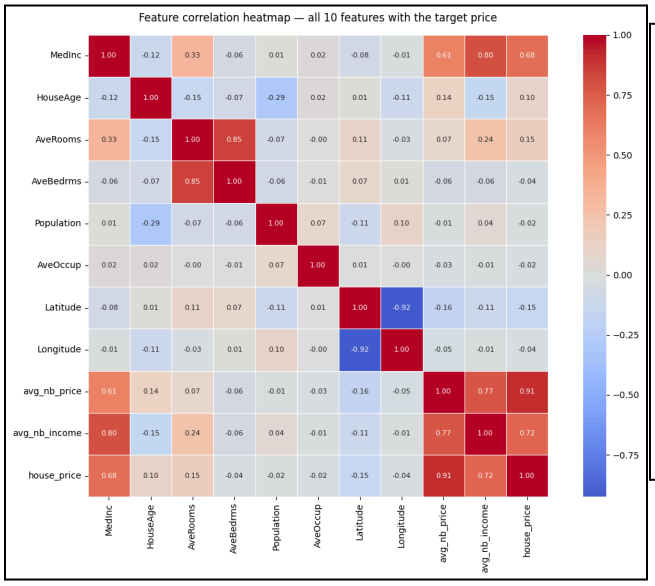
A second neighbourhood feature, avg_nb_income, was introduced to complement avg_nb_price. While avg_nb_price reflects nearby house values, avg_nb_income captures the earning level of residents, giving a different but related view of neighbourhood quality.

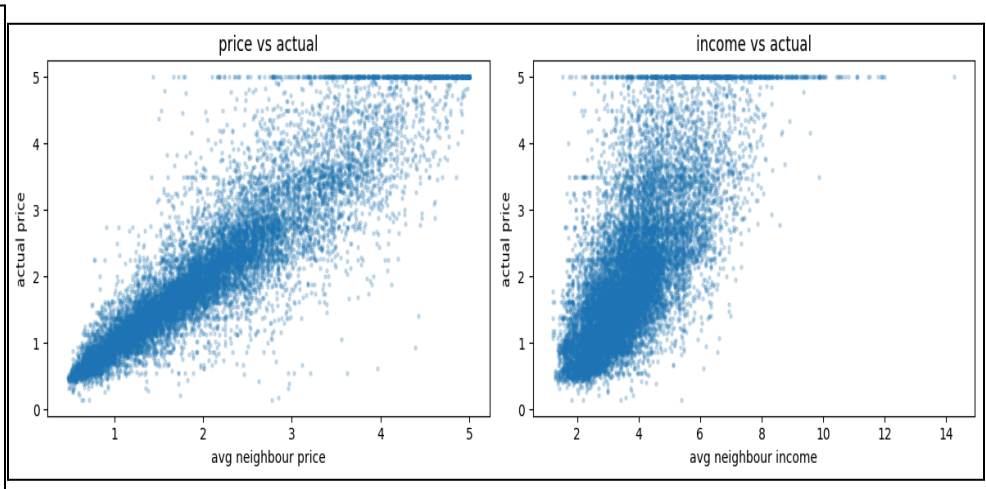
It was constructed using the same k nearest neighbours approach (k = 10, Euclidean distance on latitude and longitude), so the same neighbour sets were used. For each point, the average income of its 10 nearest neighbours was calculated using training data only, ensuring no data leakage.

Both features are strongly correlated with house price (avg_nb_price: 0.91, avg_nb_income: 0.72) and also correlate with each other (0.77), indicating they are related but not redundant. Each captures a distinct aspect of the local area, so including both is justified.

The two features were then added to the dataset and scaled using a StandardScaler fitted on the training data, resulting in a final 10 feature input for Model 2.

Correlation Analysis



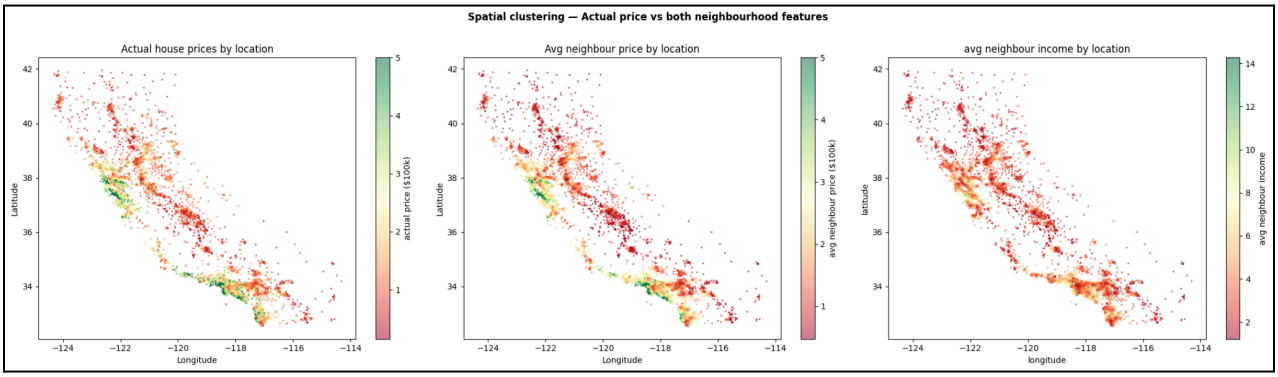


The bottom row of the heatmap on the left shows correlations with house price. avg nb price has the strongest correlation at 0.91, higher than median income at 0.68, making it the most informative feature. avg nb income is also strong at 0.72. The two neighbourhood features correlate at 0.77, so they are related but not redundant. avg nb price reflects local house values, while avg nb income shows the economic profile of the area, meaning both add useful information. In contrast, latitude and longitude have weak correlations with price at −0.15 and −0.04, showing that raw coordinates are far less informative than the neighbourhood features that we derived from them.

Zooming in on only our two additional features, the scatter plots on the right show both features correlate well with the actual house price (0.91 and 0.72) which confirms they're both useful. The difference in correlation values shows they're not just measuring the same thing and that income adds something extra on top of price.

Spatial patterns

All three maps show consistent spatial patterns, with higher prices concentrated along the coast, particularly near the Bay Area and Los Angeles, and lower prices inland. The neighbourhood features closely mirror this distribution, indicating that relative location to nearby properties is highly predictive of price, with avg_nb_income providing slightly smoother but still complementary information due to its lower correlation.



Model 2: The improved neural network

Model 2 is a feedforward neural network trained on X train final (10 features). To ensure a fair comparison with the baseline, the same starting architecture was used: two hidden layers (64 neurons → 32 neurons), ReLU activation, Adam optimiser, MSE loss function, and a maximum of 500 training iterations.

GridSearchCV with 3-fold cross-validation was also applied to optimise three hyperparameters, using the same search grid as the baseline model: hidden layer sizes ∈ {(64,32), (100,50), (32,16)}, alpha ∈ {0.0001, 0.001,0.01}, learning rate init ∈ {0.001, 0.01}. Keeping the grid identical to the baseline means the only variable between the two models is the input feature set, making the comparison controlled and meaningful.
However, GridSearchCV made both models slightly worse on the held out test set (baseline: 0.2569 → 0.2616; Model 2: 0.1681 → 0.1711), indicating mild overfitting to the 3-fold cross-validation splits. For a fair and transparent comparison, the default (untuned) configurations are therefore used as the primary result. Table 1 below reports all four configurations for completeness.

Results

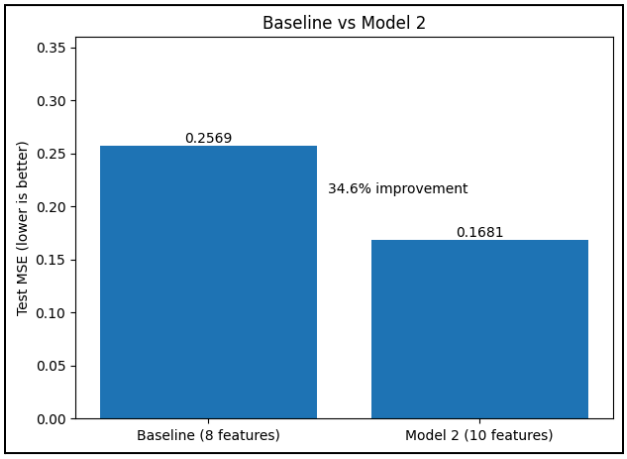
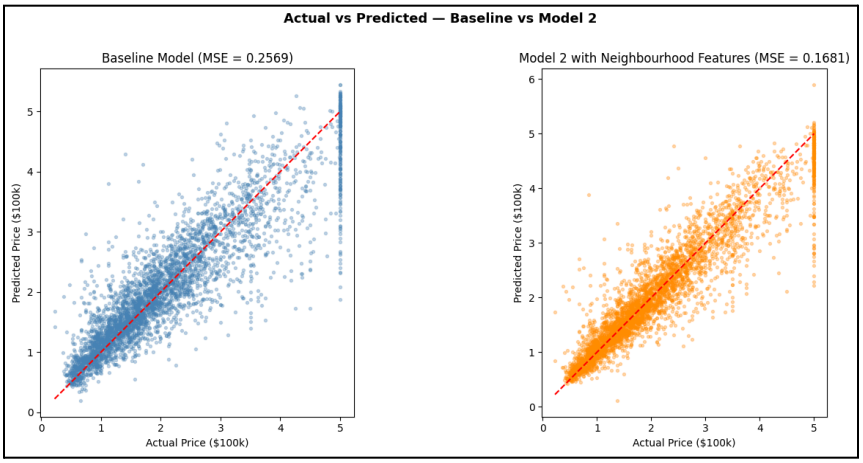


Table 1: Test set MSE across all model configurations. Primary comparison in bold.

Model	Features	Params	MSE	vs. Baseline
Baseline	**8 original**	**Default (64,32)**	**0.2569**	—
Baseline (GridSearch)	8 original	(100,50), α=0.0001	0.2616	+1.8%
Model 2	**10 features**	**Default (64,32)**	**0.1681**	**−34.6%**
Model 2 (GridSearch)	10 features	(32,16), α=0.0001, lr=0.01	0.1711	−33.4%

Model 2 achieves a test MSE of 0.1681 which is a 34.6% reduction over the baseline (0.2569). This large improvement is achieved purely by adding two neighbourhood features while keeping the same architecture.

Notably, the initial Model 2 (before grid search) already achieved 0.1681, which is lower than the optimised baseline of 0.2616. This indicates that even without hyperparameter tuning, the neighbourhood features alone create a major performance gain. The fact that GridSearchCV made both models slightly worse confirms that the neighbourhood features, not hyperparameter tuning, are responsible for the improvement. The bar chart to the above right illustrates the comparison.



The plots on the right show side-by-side scatter plots of actual versus predicted house prices for both models. Points lying on the red dashed line represent perfect predictions. Model 2's predictions visibly cluster more tightly around this line across the full price range, confirming the quantitative improvement shown in Table 1. The vertical band of points at the top of both plots reflects a known ceiling on reported house values in the dataset, which neither model can overcome.

Discussion

The results demonstrate that adding spatial neighbourhood features substantially improves the predictive performance of neural network models for housing price estimation. The baseline model, trained using only the original socioeconomic and housing features, achieved an R^2 of 0.812 and test MSE of 0.2569, confirming that these features provide useful information about local house prices. However, the enhanced model incorporating the two new neighbourhood-based features achieved a test MSE of 0.1681, a 34.6% improvement. This substantial gain emphasises the importance of spatial dependency in real estate markets.

The graph-derived features we introduced, namely average neighbouring house price and average neighbouring median income, enabled the model to capture local price dynamics that were absent from the original variables. There was a strong correlation

between these derived features and house prices, suggesting that nearby housing areas share similar economic and structural factors. This aligns with economic principles, as property markets are significantly influenced by local demand, accessibility, and regional development patterns. Consequently, the inclusion of neighbourhood features allowed the model to learn richer nonlinear relationships and substantially improve its performance on unseen test data.

Limitations

Despite the strong improvement in predictive performance, several limitations remain. First, the dataset represents aggregated census block groups rather than individual property transactions, limiting the model's ability to capture fine-scale price variation. Predictions therefore reflect area-level trends rather than true property values, placing a natural ceiling on achievable accuracy.

Second, the dataset imposes an upper cap on reported house values, visible as vertical clustering in the prediction plots. This prevents the model from learning variation among high-value properties, leading to systematic underperformance in this range. Log-transforming the target or using methods for censored data could address this.

The neighbourhood feature construction also has limitations. Euclidean distance ignores real-world structure such as road networks and geographical barriers. The fixed neighbourhood size of k = 10 was not optimised, despite optimal locality likely varying between urban and rural regions. Additionally, avg_nb_price is derived from training labels; while this avoids direct leakage, it introduces a dependence on spatial autocorrelation, meaning the model may partially rely on local price smoothing rather than learning underlying relationships. If constructed incorrectly, such features would introduce severe data leakage.This limits generalisability to markets where historical price data is unavailable or spatial dynamics differ from California.

Finally, hyperparameter tuning using GridSearchCV slightly worsened test performance for both models (baseline: 0.2569 to 0.2616; Model 2: 0.1681 to 0.1711), indicating the 3 fold validation eliminated a “lucky split” providing a more reliable estimate. Untuned models were retained for a fair comparison, although this may leave some performance improvements unrealised.

Possible Improvements

Tuning the value of k rather than fixing it at 10 would allow the model to adapt to varying housing densities across urban and rural areas. Using distance-weighted aggregation, where closer neighbours contribute more than distant ones, could produce more geographically precise features. Applying a log transformation to the target variable before training would reduce the impact of the price ceiling that causes vertical clustering in the prediction plots. Finally, incorporating early stopping during training would prevent potential overfitting on longer runs and reduce computation time.

Conclusion

This study demonstrates that incorporating graph derived spatial features leads to substantial improvements in neural network regression performance without requiring complex architectures or extensive hyperparameter tuning. By encoding neighbourhood relationships using k nearest neighbours, we extracted two complementary features: the average neighbouring house price and the average neighbouring median income, both of which outperform the original eight input variables in predictive power. The resulting 34.6% reduction in test mean squared error from 0.2569 to 0.1681 can be attributed entirely to these neighbourhood features, supporting the idea that local spatial dependencies are key drivers of variation in housing prices.

This result highlights an important principle in machine learning, which is that careful feature engineering informed by domain knowledge often delivers greater improvements than increasing model complexity. For applications such as real estate valuation and other spatially dependent problems, this approach shows that meaningful structure can be extracted from raw coordinates by incorporating information from nearby areas. The method is simple to implement, requiring only k nearest neighbours and basic aggregation, yet it produces a clear and significant improvement in performance, making it a practical and effective component of predictive modelling in this setting.

References

1. Buhl, N. (2024) *Train, Validation, Test Split for Machine Learning*. Encord. Available at: https://encord.com/blog/train-val-test-split/ (Accessed: 26/3/26).

2. Pace, R.K. and Barry, R. (1997) 'Sparse Spatial Autoregressions', *Statistics and Probability Letters*, 33(3), pp. 291–297.

3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.